

Testare unitară în Python

Cernea Ana

Grigorașcu Andrei-Antonio

Măciucă Emma-Iulia

Năvodaru Denisa-Ștefania

Nichita Iulia-Nicoleta

Mai, 2026

Unit test
Python



SISTEM REZERVARE A LOCURILOR DIN AVION

*.rezervă_loc(rând,
loc, vârstă,
are_bagaj)*

REZERVARE LOC

Metodă utilizată pentru a rezerva locul specificat în avion pentru un anumit pasager cu specificații de vârstă și posesie bagaj de cală, ce returnează prețul final al biletului.

*.anulare_rezervare(
rând, loc)*

ANULAREA UNEI REZERVĂRI

Metodă de determină anularea locului specificat din avion, ce returnează valoarea True dacă operația s-a realizat cu succes sau False dacă locul este deja liber.

*.este_loc_disponibil
(rând, loc)*

VERIFICARE DISPONIBILITATE LOC

Metodă ce verifică dacă locul specificat este disponibil (liber), ce returnează True dacă locul este liber sau False dacă este ocupat.

.locuri_disponibile()

AFIȘARE LISTĂ LOCURI DISPONIBILE

Metodă ce returnează lista tuturor locurilor libere ca tupluri (rând, loc).

.nr_locuri_disponibile()

AFIȘARE NUMĂR LOCURI DISPONIBILE

Metodă ce afișează numărul de locuri disponibile (libere) din avion.

TESTARE FUNCȚIONALĂ

Datele de test sunt generate pe baza **specificației (cerințelor) programului**, structura programului ne jucând niciun rol. Au fost folosite tehnici precum determinarea claselor de echivalență și analiza valorilor de frontieră pentru alegerea eficientă a datelor de test.

Librărie Python utilizată: *pytest*

DETERMINAREA CLASELOR DE ECHIVALENȚĂ

Metodă	Parametru	Valoare parametru
.rezervă_loc(rând, literă_loc, vârstă, are_bagaj_cală)	rând	business (1-2), economy (3-10), valoare invalidă
	literă_loc	valoare validă (A-F), valoare invalidă
	vârstă	infant (0-1), copil (2-12), adult (13-59), senior (>=60), valoare invalidă
	are_bagaj_cală	DA, NU, valoare invalidă
.anulează_rezervare(rând, literă_loc)	rând	business (1-2), economy (3-10), valoare invalidă
	literă_loc	valoare validă (A-F), valoare invalidă

*au fost efectuate teste și pentru metoda .este_loc_disponibil(rând, literă_loc), similar metodei .anulează_rezervare(rând, literă_loc)

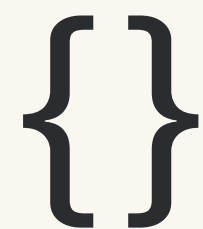
TESTARE STRUCTURALĂ

Cazurile de test sunt proiectate pe baza **structurii interne a codului sursă**, fără a lua în considerare specificația (cerințele) programului.

Pentru a utiliza metode structurale de testare, programul poate fi reprezentat sub forma unui graf orientat numit graful de flux de control (*Control flow graph* abreviat **CFG**).

Librărie Python utilizată: *unittest*

TESTARE STRUCTURALĂ



**ACOPERIRE LA
NIVEL DE
INSTRUCȚIUNE**

fiecare instrucțiune
(nod al grafului) este
parcursă măcar o dată



**ACOPERIRE LA
NIVEL DE
DECIZIE**

fiecare ramură a
grafului este parcursă
măcar o dată



**ACOPERIRE LA
NIVEL DE
CONDIȚIE**

fiecare condiție
individuală dintr-o
decizie să ia atât
valoarea
adevărat, cât și fals

STATEMENT, BRANCH, CONDITION COVERAGE



Determinare date de test pentru metoda (.anulează_rezervare(rând, literă_loc))

COD SURSĂ	INSTRUCȚIUNE	DECIZIE	CONDIȚIE
if not isinstance(rand, int) or isinstance(rand, bool): raise TypeError("Randul trebuie sa fie un numar intreg")	tip invalid rând	tip invalid rând	rând="1", rând=True
if not (1 <= rand <= self.NR_RANDOMURI): raise ValueError(f"Rand invalid: {rand}.")	valoare invalidă rând	valoare invalidă rând	rând=0, rând=11
if not self.locuri_ocupate[rand - 1] [coloana]: return False	loc deja liber	loc deja liber	loc deja liber

*au fost realizate teste similare pentru toate metodele clasei sistemrezervareavion

MC/DC COVERAGE

(fiecare condiție poate influența independent rezultatul deciziei din care face parte)

Demonstrație (D1 preluată din metoda `.rezervă_loc(rând, literă_loc, vârstă, are_bagaj)`):

TEST	C1: not isinstance(rand, int)	C2: isinstance(rand, bool)	D1: C1 or C2
R1: rand="1"	True	False	True
R2: rand=True	False	True	True
R12: rand=1	False	False	False

Perechi MC/DC:

- C1 este demonstrată prin R1 și R12: C2 rămâne False, iar D1 se schimbă din True în False.
- C2 este demonstrată prin R2 și R12: C1 rămâne False, iar D1 se schimbă din True în False.

CIRCUITE INDEPENDENTE

`(.rezervă_loc(rând, literă_loc, vârstă, are_bagaj_cală))`

Tip invalid rând

(tipul introdus nu este number)

Tip invalid literă_loc

(tipul introdus nu este literă)

Tip invalid vârstă

(tipul introdus nu este number)

Tip invalid are_bagaj_cală

(tipul introdus nu este boolean)

Locul ales provoacă dezechilibru

(diferența între nr. locuri stânga și nr. locuri dreapta este >3)

Valoare invalidă rând

(valoarea introdusă nu este între 1 și 6)

Valoare invalidă literă_loc

(valoarea introdusă nu este între A și F)

Valoare invalidă vârstă

(valoarea introdusă este negativă)

Locul ales este ocupat

Rezervare validă

Unit test
Python



*abordare similară pentru metodele `.anulează_rezervare(rând, literă_loc)` și `.locuri_disponibile(rând, literă_loc)`

MUTATION TESTING

Librărie Python utilizată: *cosmic-ray*

Avantaje cosmic-ray:

- acoperire mai mare (generarea mai multor mutații potențiale)
- detecție extinsă a mutantilor (izolează și elimină automat mutațiile care generează erori sau care blochează execuția)
- reducerea timpului de rulare prin execuție paralelă
- fișier de configurare flexibil

MUTATION TESTING

Modificări la clasa originală:

- modificarea pragului pentru echilibru
- modificarea prețurilor
- număr de rânduri/coloane alterat

Teste folosite pentru a omorî mutanții: funcționale, de validare, boundary testing, parametrizate, pentru mutanți specifici

Rezultate:

- inițial 23 teste: 136/459 mutanți rămași în viață
- 51 teste: 0 mutanți rămași în viață

Multumim!

Mai 2026

Unit test
Python

